

Proprioceptive State Machines: Fibonacci-Radix Streaming Computation with Structural Calibration

Anonymous

Submission

ICLR 2027

Double-blind review

anonymous

Abstract—We present the Fibonacci-radix Streaming Virtual Machine (FSVM), a deterministic state machine that processes boolean streams at $O(1)$ per bit with bounded heap allocation. The FSVM detects adjacency violations in Zeckendorf-coherent representations, emitting dilation events that retrospectively rescale semantic indices without rewriting data. We report measurements on how calibration parameters—adjacency width and marker threshold—affect sensitivity profiles across differently-calibrated FSVM instances. Experiments on boolean streams derived from natural language, source code, and synthetic patterns show that a 3×3 array of FSVMs with distinct (width, threshold) configurations produces different event orderings. We report these results without claiming an independent multi-axis basis: the width axis is a nested threshold on bit-run length rather than a free parameter, and the threshold axis is untested on the corpora used here because no markers fire. At ≈ 30 ns per input bit for the core state machine on commodity hardware, the FSVM operates orders of magnitude faster than learned tokenization pipelines while requiring no vocabulary, no training data, and no language-specific preprocessing.

Index Terms—streaming algorithms, tokenization, state machines, Fibonacci representations, Zeckendorf coding

I. INTRODUCTION

Modern language models depend on learned tokenizers such as byte pair encoding [1] and SentencePiece [2] that impose a discrete vocabulary on inherently continuous linguistic structure. This creates three problems: (1) the vocabulary is language-specific and corpus-dependent, (2) tokenization is a lossy discretization that discards sub-token structure, and (3) the tokenizer must be trained before the model, adding a preprocessing stage that is tightly coupled to downstream performance.

Byte-level models [3], [4], [5], [6] eliminate the tokenizer but pay a quadratic attention cost on sequences that are three to five times longer than subword sequences. Structured state space models [7], [8], [9], [10] achieve linear-time sequence processing through learned state transitions, but operate on token embeddings rather than raw signal.

We propose a different approach: analog sensing via streaming state machines. Instead of learning a vocabulary, we deploy an array of simple state machines—each calibrated with different geometric parameters—that detect structural features in a boolean stream. The “token” is which detectors are active and at what rate, not a symbol from a learned set. This draws on the biological principle of cochlear frequency analysis [11], where hair cells with different physical spacings

along the basilar membrane resonate at different frequencies without any discrete frequency vocabulary. We do not claim that this replaces tokenization for NLP tasks at present; rather, we establish the computational primitive and calibration mechanism that would be required.

Our contributions are:

- The **FSVM**: a Fibonacci-radix streaming state machine with $O(1)$ per-bit complexity and bounded heap allocation, with built-in Zeckendorf error correction. Benchmarked at ≈ 30 ns/op for the core state machine on commodity hardware.
- **Structural calibration**: measurements of how varying a geometric parameter (adjacency width) changes event rates and class orderings. Width is an ordered threshold on bit-run length; it shifts which class dominates but is not an independent axis. The marker threshold axis remains untested on current corpora.
- **The transponder array**: an architecture in which differently-calibrated FSVMs process the same bitstream in parallel, producing a multi-dimensional structural signal that replaces discrete tokenization.

II. THE FIBONACCI-RADIX STREAMING VIRTUAL MACHINE

A. State and Semantics

The FSVM is a deterministic streaming state machine operating on an unbounded boolean sequence $x_0, x_1, \dots$. The state consists of:

- $r \in \mathbb{N}$: the dilation exponent (count of retrospective dilation events)
- $W \in \{0, \dots, 63\}$: a 6-bit hexagram window over recent logical bits
- $\text{lastBit} \in \{0, 1\}$: the most recently observed bit
- $\text{zeroRun} \in \mathbb{N}$: current consecutive-zero run length
- $\text{sketch} \in \{0, 1\}^{64}$: a Zobrist state fingerprint (XOR-folded)
- $\text{Seeds} = (s_0, s_1)$: a per-instance pair of 64-bit Zobrist constants

The state occupies 96 bytes. It does not grow with stream length.

B. Step Function

On each input bit $b \in \{0, 1\}$, the FSVM performs exactly:

- 1) **Zero-run update:** If $b = 0$, increment zeroRun. If zeroRun crosses a sparse threshold (default: powers of two ≥ 8), emit a MARKER event.
- 2) **Adjacency detection:** If lastBit = 1 and $b = 1$, emit DILATE and increment r .
- 3) **Window update:** $W \leftarrow ((W \ll 1) \mid b) \bmod 64$
- 4) **Zobrist fold:** sketch $\leftarrow$ sketch $\oplus$ (Seeds[b] + $W \cdot 0 \times 9E3779B97F4A7C15$)

The multiplier on the window term is load-bearing. W takes only 64 values, so adding it directly to a seed perturbs just the low bits and leaves the upper 56 bits a function of the bit counts alone; a nominal 64-bit sketch then admits at most $4 \times 256 = 1024$ distinct values. Multiplying by an odd constant spreads each window value across the full word.

Each operation is a fixed sequence of comparisons, arithmetic, and bitwise operations. No loops, no recursion, no dynamic dispatch. The total operation count is bounded by 20 primitive operations per input bit, establishing $O(1)$ worst-case complexity.

C. Dilation Semantics

When the FSVM observes adjacent 1-bits, it increments the dilation exponent r . Conceptually, the stream is retrospectively rescaled as if a zero were inserted between the adjacent positions:

$$D(s) = s_0 0 s_1 0 s_2 0 \dots 0 s_{\{n-1\}} \quad (1)$$

But no data is rewritten. The dilation exponent r counts the minimum number of zero-insertions needed to make the observed stream Zeckendorf-coherent—free of adjacent 1-bits. Under this interpretation, effective index i would shift to $i \ll r$ if the stream were rewritten; the FSVM never performs this expansion, using r only as a structural signal.

This connects to Zeckendorf’s theorem [12], independently proven by Lekkerkerker [13], which establishes that every positive integer has a unique representation as a sum of non-consecutive Fibonacci numbers. The FSVM treats Zeckendorf coherency as a streaming invariant: DILATE events mark violations, and r measures the deviation from coherency.

D. The Zobrist Sketch

The sketch provides a 64-bit state fingerprint updated by one XOR per input bit. It inherits its mechanism from Zobrist hashing [14], originally developed for incremental board-position hashing in game-playing programs, and from the broader class of rolling and incremental hash functions [15] that maintain a running fingerprint over streaming input.

The FSVM’s fold differs from classical Zobrist hashing in two ways: (1) the seed table has only two entries (one per bit value) rather than one per board position, and (2) the window state W is folded in via addition of a spread multiple, breaking XOR commutativity and making the sketch order-sensitive despite using XOR accumulation.

The fold is not a universal hash and carries no cryptographic collision guarantee. Because accumulation is pure XOR over a 128-symbol alphabet (2 bit values $\times$ 64 window states), the sketch is parity-limited on degenerate input: an all-zero stream leaves W pinned at zero and yields only two

distinct sketches across all lengths. Empirically, roughly 1.5% of random streams of length 1–200 collide. The sketch should be read as a coherence fingerprint, not an identifier.

The sketch provides: (a) cheap divergence detection between transponders processing the same input, (b) convergence detection when the sketch stabilizes across windows, and (c) state identity fingerprinting. It does not provide class identification or semantic encoding—those belong in the analytical layer.

E. Complexity

TABLE 1: MEASURED ON AMD EPYC 7763, Go 1.25, `-BENCHMEM -COUNT=3`. THE REPORTED BYTE FIGURE IS A FRACTIONAL AVERAGE OVER THE EVENT ALLOCATIONS: STEP REPORTS 0 ALLOCS/OP BUT 3 B/OP AMORTIZED, BECAUSE UP TO ONE 16-BYTE EVENT IS ALLOCATED PER EMITTED EVENT AND DIVIDED ACROSS THE BITS IN THE BENCHMARK PATTERN.

Component	Time	Space	Allocs
FSVM Step	≈ 30 ns	96 B	0
StepV2	≈ 35 ns	96 B	0
BitRope Append	≈ 15 ns	$O(n)$ amort.	0
Array Step (k transponders)	$O(k)$	$O(k \times 96)$	$O(k)$

The FSVM’s $O(1)$ complexity is worst-case, not amortized. No input pattern triggers expensive fallback behavior. The bounded-allocation property holds because: (1) the State is passed by value (96 bytes), (2) the event slice uses append with bounded growth (at most 2 events per step), and (3) no interfaces, closures, or channels are involved in the core path.

Scope. These figures describe the core state machine only. The higher-level `session.ProcessBits` ingest path, which drives seven display extensions, is roughly two orders of magnitude slower per bit and allocates per bit; it now defers display rendering to an explicit `RefreshOutputs()` call. Benchmark the component you actually use.

III. STRUCTURAL CALIBRATION

A. The Cochlear Analogy

In the mammalian cochlea, hair cells at different positions along the basilar membrane have different physical spacings, causing them to resonate at different frequencies [11]. There is no “frequency vocabulary”—frequency decomposition emerges from sensor geometry.

The fibrotransponder array applies this principle. Each transponder is an FSVM instance with specific geometric parameters:

- **Adjacency width** $w \in \{1, 2, 3\}$: how many trailing bits must be 1 for the adjacency detector to fire
- **Marker threshold**: the zero-run length at which markers are emitted (powers of 2, powers of 3, or linear)

All transponders share the same Zobrist seed table. Only geometry differs. This distinction—seeds label trajectories; geometry changes sensitivity—was established by controlled experiment.

B. First Axis: Adjacency Width

Varying adjacency width changes which input structures trigger DILATE events. At $w = 1$, any consecutive 1-bits trigger dilation; at $w = 3$, three consecutive 1-bits are required.

TABLE II: DILATE RATE PER (WIDTH $\times$ CLASS). BOLD = HIGHEST RATE PER ROW. DIFFERENT WIDTHS ARE MOST SENSITIVE TO DIFFERENT INPUT CLASSES.

Width	Prose	Code	Synthetic	Most sensitive
$w = 1$	0.1992	0.1763	0.0313	prose
$w = 2$	0.0582	0.0609	0.0000	code
$w = 3$	0.0079	0.0150	0.0000	code

Two falsification checks passed: (A) dil-rate ordering is not monotonic across widths ($w=1$ ranks prose > code; $w=2$ and $w=3$ rank code > prose), and (B) the most-sensitive class changes with width. This establishes that structural calibration produces different detectors, not rescaled copies of one detector.

C. Second Axis: Marker Threshold

The marker threshold defines when zero-run markers fire. We test three threshold families: powers of two ≥ 8 (default), powers of three ≥ 9 , and linear (every k zeros).

The original corpora produce zero markers because they lack long zero runs. We expand the corpora by interleaving original content with zero-padded blocks (64–128 bytes of zeros), creating diverse zero-run distributions.

TABLE III: MARKER RATE PER (THRESHOLD $\times$ CLASS) AT $w = 1$. DIFFERENT THRESHOLDS PRODUCE DIFFERENT CLASS ORDERINGS. THIS HOLDS AT ALL THREE WIDTHS.

Threshold	Prose+zeros	Code+zeros	Mixed	Ranking
pow2 ≥ 8	0.000817	0.000762	0.000496	prose > code > mixed
pow3 ≥ 9	0.000467	0.000476	0.000297	code > prose > mixed
lin 4	0.052054	0.066546	0.073236	mixed > code > prose
lin 8	0.007470	0.012186	0.001850	code > prose > mixed
lin 12	0.004902	0.008092	0.001222	code > prose > mixed

At every fixed width, different threshold families produce different class orderings.

D. Independence Claim (withdrawn)

Withdrawn. The previous version of this section asserted an “Independence Theorem”: that width and threshold are orthogonal structural axes spanning a 2-dimensional parameter space. That claim does not survive closer analysis and has been removed.

Two reasons:

- 1) **Width is a nested threshold, not a free axis.** The widths test for 1-runs of length $\geq 2, 3, 4$ respectively, so the event sets are nested ($\rightarrow$ for runs of length $\geq$

$4 \subset \text{runs} \geq 3 \subset \text{runs} \geq 2$). Dilation counts are non-increasing in width for every input, by construction; TestStructuralCalibration verifies this on 2000 randomized streams. The class-ranking shift at $w = 1$ vs $w = \frac{2}{w} = 3$ is a consequence of that nesting, not evidence of independent detectors.

- 2) **The threshold axis is untested on these corpora.** Marker counts are zero across all nine (width, threshold) configurations on all three classes, so no independence conclusion can be drawn from marker orderings. TestSecondAxisCalibration asserts this precondition explicitly.

What remains supported: at fixed width, different threshold families do produce different class orderings on the observed marker counts, and the width axis shifts which class dominates the dilation rate. Neither observation establishes orthogonality.

IV. THE TRANSPONDER ARRAY

A. Architecture

A transponder array consists of k FSVM instances, each with distinct geometric calibration, processing the same input bitstream in parallel. Each transponder runs at the core FSVM speed (≈ 60 ns). The array processes in $O(k)$ time per input bit.

The output of the array is a k -dimensional vector of (dilation rate, marker rate, sketch) triples, updated every bit. This vector is the analog equivalent of a token embedding—but it is extracted by geometry, not learned from data.

B. Potential Application: Input Layer Replacement

The following is a proposed architecture, not an implemented system. It is included to clarify the design intent behind the transponder array and to motivate the calibration mechanism developed in Section 3.

In a conventional pipeline, text passes through: raw bytes $\rightarrow$ BPE tokenization $\rightarrow$ embedding lookup $\rightarrow$ positional encoding $\rightarrow$ transformer. The FSVM array could replace the first three stages: raw bytes $\rightarrow$ UTF-8 bitstream $\rightarrow$ FSVM array $\rightarrow$ structural signal $\rightarrow$ transformer.

In this hypothetical architecture, the “vocabulary” becomes the array calibration, the “context window” becomes the dilation history, and positional encoding is subsumed by the Fibonacci indexing inherent in dilation. Whether this produces useful representations at scale remains an open question: Section 6 reports that byte-frequency features currently outperform FSVM features for text classification at small corpus scales.

C. Convergence and Halting (Proposed)

Current transformer models halt extrinsically—by reaching a maximum token count or generating an end-of-sequence token. The FSVM array suggests a possible alternative: intrinsic convergence through homeostatic settling. The system would settle when (1) DILATE rate approaches zero (Zeckendorf coherency achieved), (2) the Zobrist sketch stabilizes (no state transitions detected), and (3) marker frequency drops below threshold (silence).

This is analogous to a ball rolling to the bottom of a bowl—the system does not “finish” in the stepwise sense but reaches a quiescent state. Convergence guarantees for this feedback loop are not proven; empirical validation and formal analysis remain future work.

V. RELATED WORK

Tokenization. Subword tokenization [1], [2] remains the dominant paradigm for preparing text input to neural models. Both BPE and SentencePiece learn a discrete vocabulary from training data, creating a language-specific preprocessing stage. Byte-level models [3], [4], [5] eliminate the vocabulary but incur longer sequences and higher attention costs. MEGABYTE [6] mitigates this with a multi-scale architecture that processes million-byte sequences at competitive performance with subword models. The FSVM differs from all of these: it extracts structural features from a boolean stream using a fixed state machine, with no learned vocabulary and no neural preprocessing.

Streaming algorithms. The FSVM’s $O(1)$ per-step processing connects to the streaming algorithms literature established by Alon et al.’s AMS sketch [16], the Count-Min Sketch [17], and the Count Sketch [18], surveyed by Muthukrishnan [19]. These randomized data structures approximate aggregate statistics over streams with bounded memory. The FSVM shares the commitment to bounded work per symbol but is deterministic and tracks structural properties rather than frequency statistics.

State space models. Structured state space models [7], [8] achieve linear-time sequence processing through continuous-time state transitions with learned parameters. RWKV [9] combines RNN efficiency with transformer-scale parallel training, and Hyena [10] replaces attention with learned long convolutions at subquadratic cost. The FSVM shares the state-based paradigm but operates on raw bits with discrete, bounded state (96 bytes) and geometry-defined (not learned) transitions.

Biological signal processing. The transponder array draws on cochlear frequency analysis [11], where frequency decomposition emerges from sensor geometry rather than discrete frequency bins. The auditory scene analysis framework of Bregman [20] provides the theoretical grounding for streaming-based source segregation. The scattering transform [21] applies a similar principle to visual signals, constructing translation-invariant representations through cascaded wavelet decompositions without learned parameters.

VI. LIMITATIONS

The FSVM is proven on boolean streams. Its application to natural language processing is proposed but not yet demonstrated. In a 5-fold cross-validation classification experiment on small corpora (≈ 1 KB per class), byte-frequency features (7 dimensions) achieved 100% accuracy on prose/code/structured discrimination while FSVM array features (24 dimensions) achieved 66.7%, suggesting that the current FSVM signal is weaker than simple byte statistics at this scale.

Larger corpora and different task formulations may be needed to realize the analog tokenization hypothesis.

The transponder array’s calibration parameters (adjacency width, marker threshold) are currently hand-set, not learned. The second-axis experiment uses expanded corpora with artificial zero padding; validation on real-world streaming data remains future work.

The Zobrist sketch is not a universal hash. Accumulation is pure XOR over a 128-symbol alphabet (2 bit values times 64 window states), which makes the sketch parity-limited: an all-zero stream leaves the window pinned at zero and admits only two distinct sketches across all lengths, and roughly 1.5% of random streams of length 1–200 collide. It serves as a practical state fingerprint for coherence and divergence, not a cryptographic commitment or a class identifier.

The proprioceptive feedback loop (measuring DILATE rate, adjusting sensor geometry, observing signal change) is described but not implemented. Convergence guarantees for the feedback loop are empirical, not proven.

The analog tokenization hypothesis — that structurally calibrated FSVM arrays can supplant learned tokenization for NLP tasks — is empirically falsifiable. A concrete falsification condition is: on a diverse corpus suite totaling >1 MB of natural text, after systematic calibration across widths $\{1,2,3\}$ and threshold families, the FSVM array signal fails to exceed simple non-learned baselines (byte-frequency histograms, character n -grams) on a battery of standard text classification benchmarks. The current evidence remains preliminary; this condition is untested at scale.

VII. CONCLUSION

We have presented the FSVM, a Fibonacci-radix streaming state machine that processes boolean streams at $O(1)$ per bit with bounded allocation. We measured how structural calibration—varying geometric parameters rather than hash seeds—changes event rates and class orderings. We explicitly do not claim an independent detector basis: the width axis is a nested threshold on bit-run length, and the marker threshold axis is untested on the corpora used here. An earlier draft asserted orthogonality between the two axes; that claim is withdrawn (see the Independence Claim section).

The FSVM core operates at ≈ 30 ns per input bit on commodity hardware (AMD EPYC 7763), orders of magnitude faster than learned tokenization pipelines. It requires no vocabulary, no training data, and no language-specific preprocessing. The transponder array is motivated by the biological principle of cochlear frequency decomposition, though as shown above its current calibration axes are ordered thresholds rather than independent resonances.

The path forward involves testing the array on larger corpora, implementing the proprioceptive feedback loop, and exploring task formulations where bit-level temporal dynamics provide signal that byte-frequency statistics cannot. Whether the analog tokenization hypothesis holds at scale remains an open question—but the computational primitive and the calibration mechanism are now established.

VIII. APPENDIX

A. Benchmark Commands

All benchmarks reproduce with:

```
go test -bench=. -benchmem ./internal/fsvm
go test -bench=. -benchmem ./internal/
bitrope
go test -run TestSecondAxisExpanded -v ./
internal/transponder
```

B. Reproducibility

The complete implementation, test suite, and experiment infrastructure are available at the project repository. All experiments use deterministic FSVM processing with fixed seed tables. No randomness is involved in the core state machine or structural calibration experiments.

REFERENCES

- [1] R. Sennrich, B. Haddow, and A. Birch, “Neural Machine Translation of Rare Words with Subword Units,” in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016, pp. 1715–1725. doi: 10.18653/v1/P16-1162.
- [2] T. Kudo and J. Richardson, “SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*, 2018, pp. 66–71. doi: 10.18653/v1/D18-2012.
- [3] L. Xue *et al.*, “ByT5: Towards a token-free future with pre-trained byte-to-byte models,” *Transactions of the Association for Computational Linguistics (TACL)*, vol. 10, pp. 291–306, 2022, doi: 10.1162/tacl_a_00461.
- [4] N. Kalchbrenner, L. Espeholt, K. Simonyan, A. van den Oord, A. Graves, and K. Kavukcuoglu, “Neural Machine Translation in Linear Time,” in *arXiv preprint*, 2016.
- [5] J. H. Clark, D. Garrette, I. Turc, and J. Wieting, “CANINE: Pre-training an Efficient Tokenization-Free Encoder for Language Representation,” in *Transactions of the Association for Computational Linguistics (TACL)*, 2022, pp. 73–91. doi: 10.1162/tacl_a_00448.
- [6] L. Yu, D. Simig, C. Flaherty, A. Aghajanyan, L. Zettlemoyer, and M. Lewis, “MEGABYTE: Predicting Million-byte Sequences with Multi-scale Transformers,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [7] A. Gu, K. Goel, and C. Ré, “Efficiently Modeling Long Sequences with Structured State Spaces,” in *International Conference on Learning Representations (ICLR)*, 2022.
- [8] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” in *arXiv preprint*, 2023.
- [9] B. Peng *et al.*, “RWKV: Reinventing RNNs for the Transformer Era,” *Findings of the Association for Computational Linguistics: EMNLP*, 2023.
- [10] M. Poli *et al.*, “Hyena Hierarchy: Towards Larger Convolutional Language Models,” in *International Conference on Machine Learning (ICML)*, 2023.
- [11] G. von Békésy, *Experiments in Hearing*. McGraw-Hill, 1960.
- [12] E. Zeckendorf, “Représentation des nombres naturels par une somme de nombres de Fibonacci ou de nombres de Lucas,” *Bulletin de la Société Royale des Sciences de Liège*, vol. 41, pp. 179–182, 1972.
- [13] C. G. Lekkerkerker, “Representation of natural numbers as a sum of Fibonacci numbers,” *Simon Stevin*, vol. 46, 1972.
- [14] A. L. Zobrist, “A New Hashing Method with Application for Game Playing,” technical report 88, 1970.
- [15] R. M. Karp and M. O. Rabin, “Efficient Randomized Pattern-Matching Algorithms,” *IBM Journal of Research and Development*, vol. 31, no. 2, pp. 249–260, 1987, doi: 10.1147/rd.312.0249.
- [16] N. Alon, Y. Matias, and M. Szegedy, “The space complexity of approximating the frequency moments,” in *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*, 1996, pp. 20–29. doi: 10.1145/237814.237823.
- [17] G. Cormode and S. Muthukrishnan, “An improved data stream summary: The count-min sketch and its applications,” *Journal of Algorithms*, vol. 55, no. 1, pp. 58–75, 2005, doi: 10.1016/j.jalgor.2003.12.001.
- [18] M. Charikar, K. Chen, and M. Farach-Colton, “Finding Frequent Items in Data Streams,” in *Proceedings of the 29th International Colloquium on Automata, Languages and Programming (ICALP)*, 2002, pp. 693–703.
- [19] S. Muthukrishnan, “Data streams: Algorithms and applications,” *Foundations and Trends in Theoretical Computer Science*, vol. 1, no. 2, pp. 117–236, 2005, doi: 10.1561/0400000002.
- [20] A. S. Bregman, *Auditory Scene Analysis: The Perceptual Organization of Sound*. MIT Press, 1990.
- [21] J. Bruna and S. Mallat, “Invariant Scattering Convolution Networks,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 8, pp. 1872–1886, 2013, doi: 10.1109/TPAMI.2012.230.